



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology



أكاديمية كاوست  
KAUST ACADEMY

# The Art of Training Deep Neural Networks

King Khalid University (KKU)  
KAUST Academy

# Table of Contents

1. Training Lifecycle
2. Losses & Metrics
3. Hyperparameter Tuning
4. Reading & Debugging Loss Curves

# Training Lifecycle

Set the Fastest and Cheapest  
End-to-End Baseline

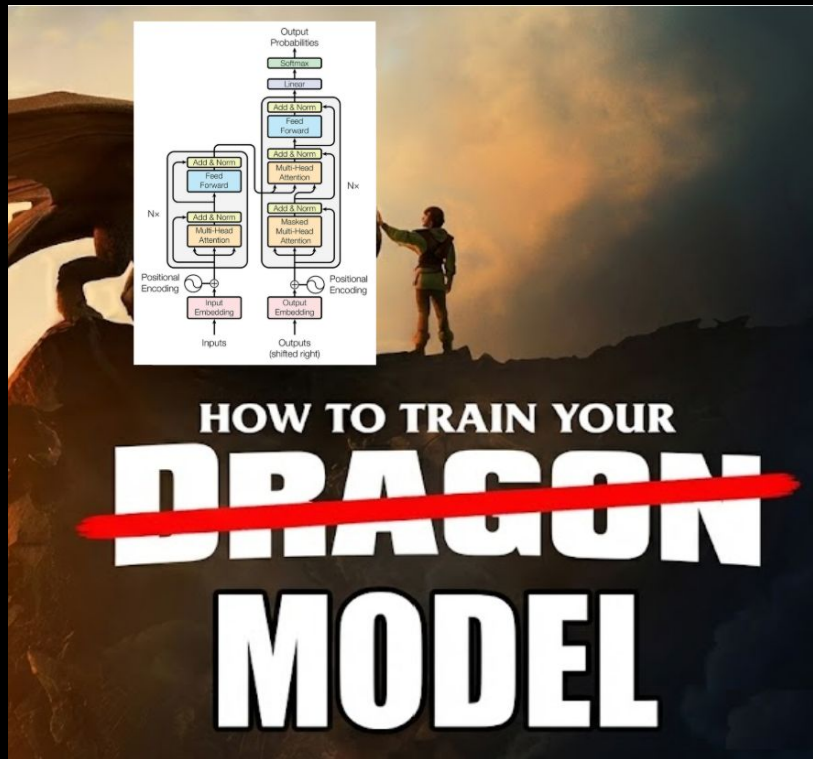
(small model, dataset, split, metric)  
→ Record the base score

Try a Strategy

(e.g. pick better loss, architecture,  
hyperparameters)

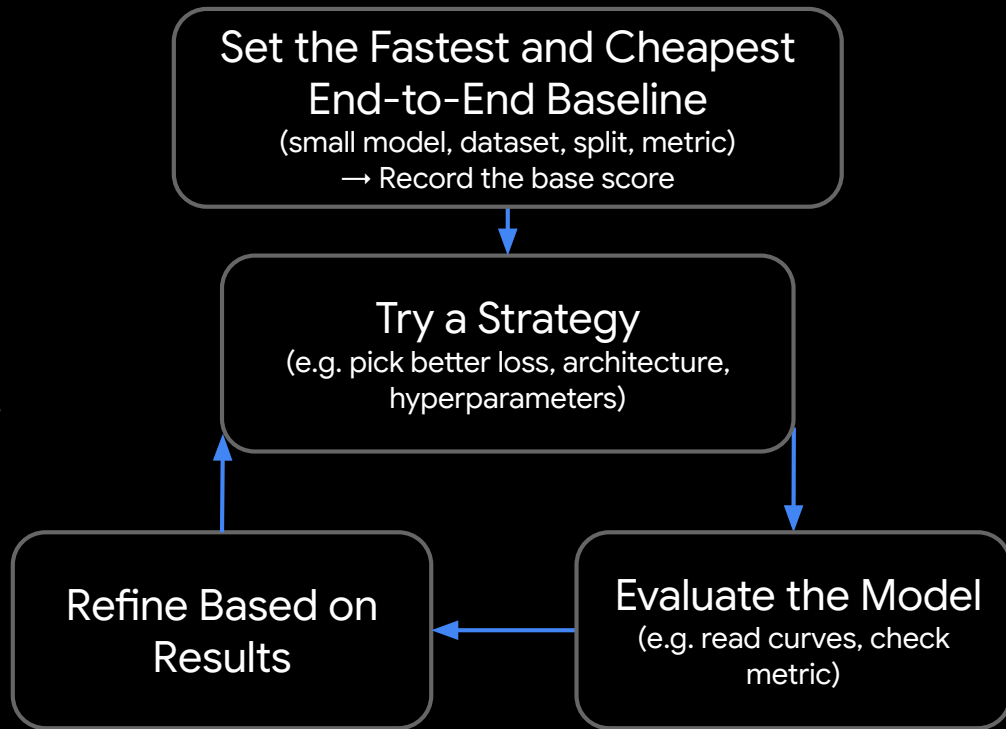
Refine Based on  
Results

Evaluate the Model  
(e.g. read curves, check  
metric)



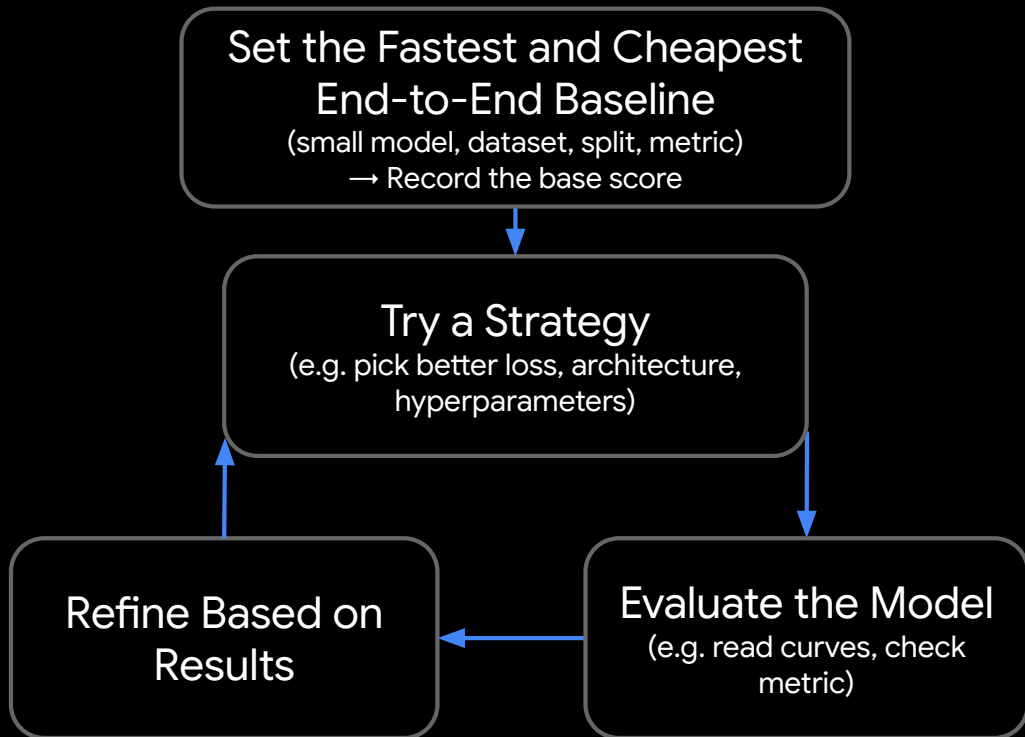
# Training Lifecycle

- Training AI models is an **iterative process** of trying, failing, and refining.
- Success depends on **empirical experimentation**, not just theory.
- Practically, we can offer guidelines on what is likely to work, but there are **no guarantees**.



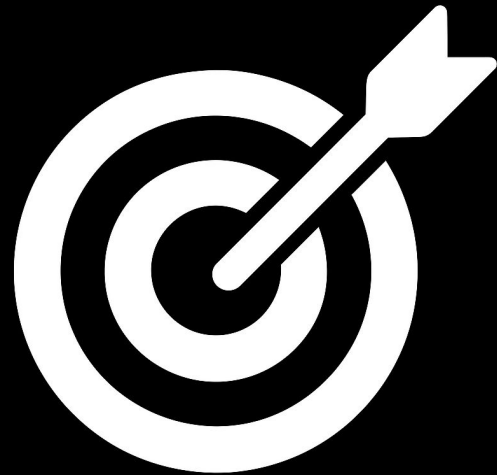
# Training Lifecycle

- In this session, we'll explore the **key components** that influence deep learning training.
- What they are, why they matter, and how to optimize them.



# Losses & Metrics

*What You Optimize Is What You Get.*



# Loss vs Metric: spot the difference

| Role   | Used during | Differentiable? | Typical examples                                |
|--------|-------------|-----------------|-------------------------------------------------|
| Loss   | ?           | ?               | Cross-Entropy, MSE, L1, Huber, Focal, Dice-Loss |
| Metric | ?           | ?               | Accuracy, F1, mAP, IoU, AUC-ROC, MAE            |



# Loss vs Metric: spot the difference

| Role   | Used during            | Differentiable? | Typical examples                                |
|--------|------------------------|-----------------|-------------------------------------------------|
| Loss   | Back-prop optimisation | Yes             | Cross-Entropy, MSE, L1, Huber, Focal, Dice-Loss |
| Metric | Validation / reporting | No              | Accuracy, F1, mAP, IoU, AUC-ROC, MAE            |

***“we often can't optimize the thing we actually care about (accuracy, IoU, F1, profit), so we optimize an auxiliary loss that is differentiable and **\*\*correlates\*\*** with it.”***



# Some Popular Losses

| Loss                               | Minimises ...                        | Used for ...                            |
|------------------------------------|--------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | ?                                    | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | ?                                    | Regression / autoencoders               |
| Dice / IoU Loss                    | ?                                    | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | ?                                    | Classification / language models (LLMs) |



# Some Popular Losses

| Loss                               | Minimises ...                        | Used for ...                            |
|------------------------------------|--------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | $L^1$ distance between points/pixels | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | ?                                    | Regression / autoencoders               |
| Dice / IoU Loss                    | ?                                    | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | ?                                    | Classification / language models (LLMs) |

*Is MAE differentiable?*



# Some Popular Losses

| Loss                               | Minimises ...                        | Used for ...                            |
|------------------------------------|--------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | $L^1$ distance between points/pixels | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | ?                                    | Regression / autoencoders               |
| Dice / IoU Loss                    | ?                                    | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | ?                                    | Classification / language models (LLMs) |

*Is MAE differentiable? No, but we can use a special type of gradient (subgradient) to minimize it.*



# Some Popular Losses

| Loss                               | Minimises ...                                                                            | Used for ...                            |
|------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels                                                     | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | $L^1$ distance between points/pixels                                                     | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | Piecewise $L^1$ & $L^2$ (quadratic near 0, linear beyond) distance between points/pixels | Regression / autoencoders               |
| Dice / IoU Loss                    | ?                                                                                        | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | ?                                                                                        | Classification / language models (LLMs) |



# Some Popular Losses

| Loss                               | Minimises ...                                                                            | Used for ...                            |
|------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels                                                     | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | $L^1$ distance between points/pixels                                                     | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | Piecewise $L^1$ & $L^2$ (quadratic near 0, linear beyond) distance between points/pixels | Regression / autoencoders               |
| Dice / IoU Loss                    | Overlap ratio between masks/boxes                                                        | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | ?                                                                                        | Classification / language models (LLMs) |



# Some Popular Losses

| Loss                               | Minimises ...                                                                            | Used for ...                            |
|------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------|
| Mean Squared Error (MSE)           | $L^2$ distance between points/pixels                                                     | Regression / autoencoders               |
| Mean Absolute Error (MAE)          | $L^1$ distance between points/pixels                                                     | Regression / autoencoders               |
| Huber (Smooth L1) Loss             | Piecewise $L^1$ & $L^2$ (quadratic near 0, linear beyond) distance between points/pixels | Regression / autoencoders               |
| Dice / IoU Loss                    | Overlap ratio between masks/boxes                                                        | Segmentation / Detection                |
| Cross-Entropy (CE) / KL divergence | Information (entropy) difference between probability distributions                       | Classification / language models (LLMs) |

# Loss Functions

***Q: How can I choose the metric / loss suitable for my problem?***

# Loss Functions

***Q: How can I choose the metric / loss suitable for my problem?***

***1- Task: Regression / Classification / Segmentation / ...***

***2- Properties:***

- Penalty behaviour*
- Magnitude effect*



# Loss Properties

## ***1- Penalty behaviour: Are all errors the same?***

- *Penalty Severity (e.g., same, power, logarithm asymptotic infinity, cap/threshold)*
- *outliers vs non-outliers.*
- *Symmetric vs asymmetric.*
- *Weighting minority classes.*

# Loss Properties

## ***2- Magnitude Effect: Does magnitude matter?***

- *Scale / relative vs absolute (MAE vs MAPE)*
- *In MAPE, values like 10 vs 11  $\approx$  100 vs 110*

# Loss Functions

*Hmm, let's talk more fundamentally...*

# Loss Functions

*Hmm, let's talk more fundamentally...*

***Q: How losses are invented? Can i create my own loss?***

# Loss Functions

***Q: How losses are invented? Can i create my own loss?***

“Designing a loss is mostly a translation problem, and the hard part is in plain language, not in math”

*Let's see an example...*

# Loss Functions

*First step: you should define the problem.*

# Loss Functions

*First step: you should define the problem.*

*Hmmm, what about building a meows-to-english translator!!*

# Loss Functions

*First step: you should define the problem.*

*Hmmm, what about building a meows-to-english translator!!*





# Loss Functions

***First step:** you should define the problem.*

*Hmmm, what about building a meows-to-english translator!!*

*Yeah looks fun, but let's try to utilize our time more effectively...*

# Loss Functions

***Let's say:*** Given some amount of products, i want to recommend some of them for the customer.

***Problem:*** Ranking a list according to some criteria.

# Loss Functions

**Second Step:** *in simple natural language terms, what is the optimal behaviour u want? (This is usually the hard to answer part!)*

# Loss Functions

**Second Step:** *in simple natural language terms, what is the optimal behaviour u want? (This is usually the hard to answer part!)*

*Items that are more relevant should appear higher in the list than less relevant ones, and getting the top of the list right matters far more than the bottom.*

# Loss Functions

***Let's break this down:***

***higher than:*** you care about relative order, not absolute scores. So, magnitude doesn't matter!

***more relevant:*** you need a ground-truth notion of relevance (labels, clicks, ratings).

***top matters more:*** the penalty is position-weighted.

# Loss Functions

**Answer:** NDCG or Precision@k

*This is why step 2 is "hard to answer": the words already determine the math!*



# Loss Functions

*Another example:*

*I want to count the people  
in this image.*





# Loss Functions

***Another example:***

*I want to count the people  
in this image.*

***What should the model  
optimize for in each part  
of the image?***





# Loss Functions

***What should the model optimize for in each part of the image?***

*1- Confirm if there is at least one object in that part of the image or no.*

*2- Ensure that object/s is a human/s (hajj)*

*3- Count each one individually*

# Loss Functions

*What should the model optimize for in each part of the image?*

*1- confirm if there is at least one object in that part of the image or no.*

*→ (Objectness)*

*2- Ensure that object/s is a human/s → (Classification)*

*3- count each one individually → (Drawing boxes)*

*→ YOLO Loss! Just count the boxes!*



# Loss Functions

*The main insight is: Think how a human would solve your problem / how the optimal solution look like, then try to convert the process into precise words and convert them to a loss.*

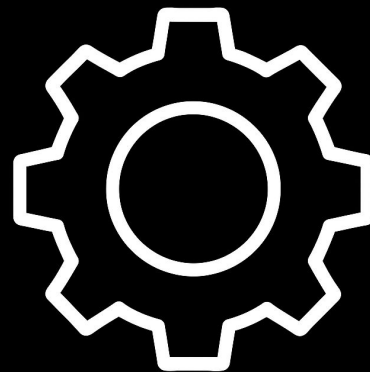


# Some Popular Composite Losses

| Scenario            | Composite Loss                                    | Purpose                           |
|---------------------|---------------------------------------------------|-----------------------------------|
| Detection (Yolo)    | $BCE_{obj} + BCE_{cls} + \lambda \cdot IoU_{reg}$ | Objectness + class + boxes        |
| Segmentation (UNet) | $CE + \lambda \cdot Dice$                         | Pixels + mask overlap             |
| Generative (VAE)    | $Reconstruction_{obj} + \beta \cdot KL$           | Rebuild + Gaussian latent         |
| Generative (GAN)    | $CE_{adv} + \lambda \cdot Perceptual$             | Realism + texture & color details |

# Hyperparameters Tuning

*Loss tells us where to climb  
Hyper-params decide how fast and which path.*



# Hyperparameters

## What are Hyper-parameters?

⇒ values you set before training (not learned).

## Role?

⇒ steer optimisation speed, capacity & generalisation.

## How to set them?

⇒ pick sensible starters → train → inspect → iterate.



# Hyperparameters Tuning

| Hyperparameter                | Quick rule-of-thumb                                                                                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optimiser                     | Usually <b>Adam/AdamW</b> works the best.                                                                                                                           |
| LR                            | <b>CNNs-based</b> : (1e-3–1e-4)-ish ↔ <b>Transformers-based</b> : (1e-5–1e-6)-ish.<br>Usually used with a <b>scheduler</b> (learning rate decay).                   |
| Batch size                    | <b>Vision</b> : (4–32)-ish ↔ <b>Text</b> : (1–16)-ish<br>keep fixed; scale only if needed.                                                                          |
| Epochs                        | <b>Vision</b> ≈ 5–300; <b>NLP</b> ≈ 1-10; <b>LLMs</b> ≈ 1 (up to 3).                                                                                                |
| Img size /<br>sequence length | prototype small (e.g. <b>img_size</b> ≈ 224*224, <b>seq_len</b> ≈ 256)<br>→ upscale when everything else is stable.                                                 |
| Backbone family &<br>size     | start tiny → scale up once pipeline is stable<br>(e.g. <b>EfficientNetV2-Small</b> → <b>EfficientNetV2-Large</b> ,<br><b>BERT-base</b> → <b>BERT-large</b> , etc.). |



# Hyperparameters Tuning

**Tip:** Scale batch size when adjusting learning rate.

But by how much?





# Hyperparameters Tuning

**Tip:** Scale batch size when adjusting learning rate.

1. SGD: Use linear scaling rule\*

$$\text{LR}_{\text{new}} = \text{LR}_{\text{old}} \times \frac{\text{Batch Size}_{\text{new}}}{\text{Batch Size}_{\text{old}}}$$

2. Adam/AdamW: Square-root scaling rule\*

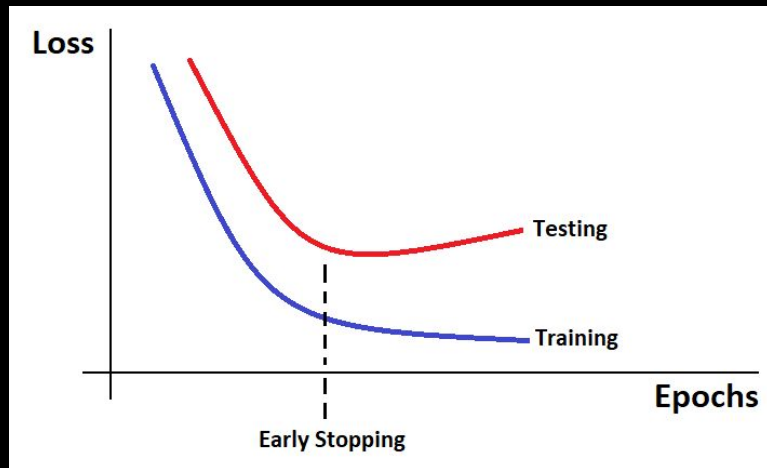
$$\text{LR}_{\text{new}} = \text{LR}_{\text{old}} \times \sqrt{\frac{\text{Batch Size}_{\text{new}}}{\text{Batch Size}_{\text{old}}}}$$

---

\*Granzio D., Zohren S., Roberts S., "Learning Rates as a Function of Batch Size: A Random Matrix Theory Approach to Neural Network Training," arXiv:2006.09092, 2020.

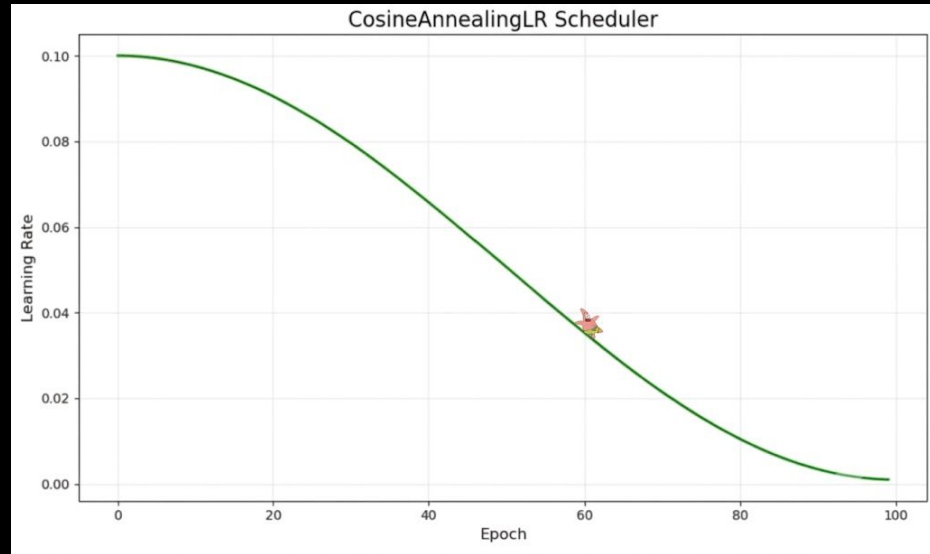
# Early Stopping

- **Goal:** stop training just after validation metric flattens or degrades.
- **How:** monitor `val-loss` / `val-metric`  $\rightarrow$  `patience=N epochs`.
- Helps auto-select **optimal epochs value**.



# Schedulers

- A **scheduler** is the rule that automatically **adjusts the learning rate** during training.
- They help speed convergence, escape plateaus, and reach a better optimum.
- Most teams now default to **cosine decay**.



# Schedulers Types

| Scheduler Type | How It Triggers                   | Typical Use-case                                                        |
|----------------|-----------------------------------|-------------------------------------------------------------------------|
| Step-based     | After every optimiser <b>step</b> | When <b>low</b> number of epochs used<br>(e.g. NLP)                     |
| Epoch-based    | After every <b>epoch</b>          | When a <b>high</b> number of epochs used<br>(e.g. Vision, Audio,...etc) |

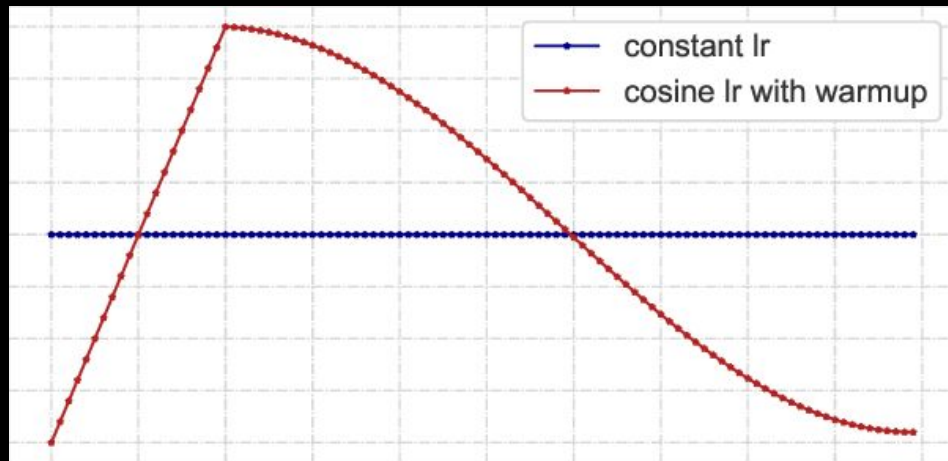
# Warm-up

- **Problem:**
  - large initial LR + random weights  
⇒ gradients explode.
  - large initial LR + Pretrained weights  
⇒ Forget previous knowledge.



# Warm-up

- **Solution:** gradually **increase** LR from 0  $\rightarrow$  base LR over some steps/epochs (or 3-5 % of total steps/epochs).
- Usually used with **Transformers**.



# Early-Stopping vs Fixed Epochs

- **Which one is better:**
  - Many epochs + Early stopping.
  - Fixed epochs + Learning rate scheduler.

# Early-Stopping vs Fixed Epochs

- **Which one is better:**

- Many epochs + Early stopping.
- Fixed epochs + Learning rate scheduler.

⇒ **Recommended:**

1. Use **early stopping initially** to discover optimal epoch range.
2. Then, set a **fixed number of epochs** and **apply a scheduler** to smoothly decay the learning rate within that range.



# Augmentations

- **Data augmentation** is generating new training examples from existing ones through various transformations.
- **Types:**

## *Random Augmentations:*

Flip, Crop, Noise,...

## *Mix Augmentations:*

MixUp, CutMix,...

- **How to choose?** do error analysis  
→ add augs that mimics real mistakes.
- **Note:** heavy augs  $\Rightarrow$  add more epochs.

# Tuning Order (practical)

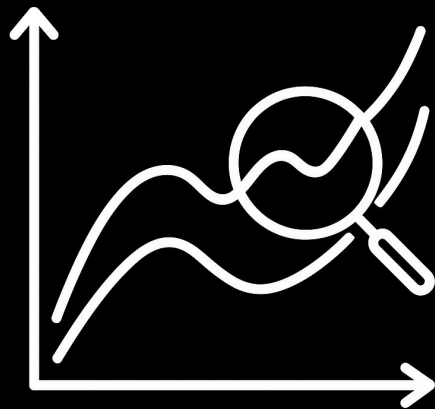
1. **LR & Epochs:** lock batch size unless GPU forces change.
2. **Scheduler:** cosine is usually the best.
3. **Augmentations:** add gradually, re-train.
4. **Model / Input complexity:** scale backbone, image size, sequence length.

# Tuning Order (practical)

- **Tips:**
  - Make one change at a time.
  - Start with small experiments, then scale up.
  - Always ensure the loss behaves normally and check for common bugs (e.g., exploding loss, NaNs, unstable curves).

# Reading & Debugging Loss Curves

*See the Signal, Catch the Bug*



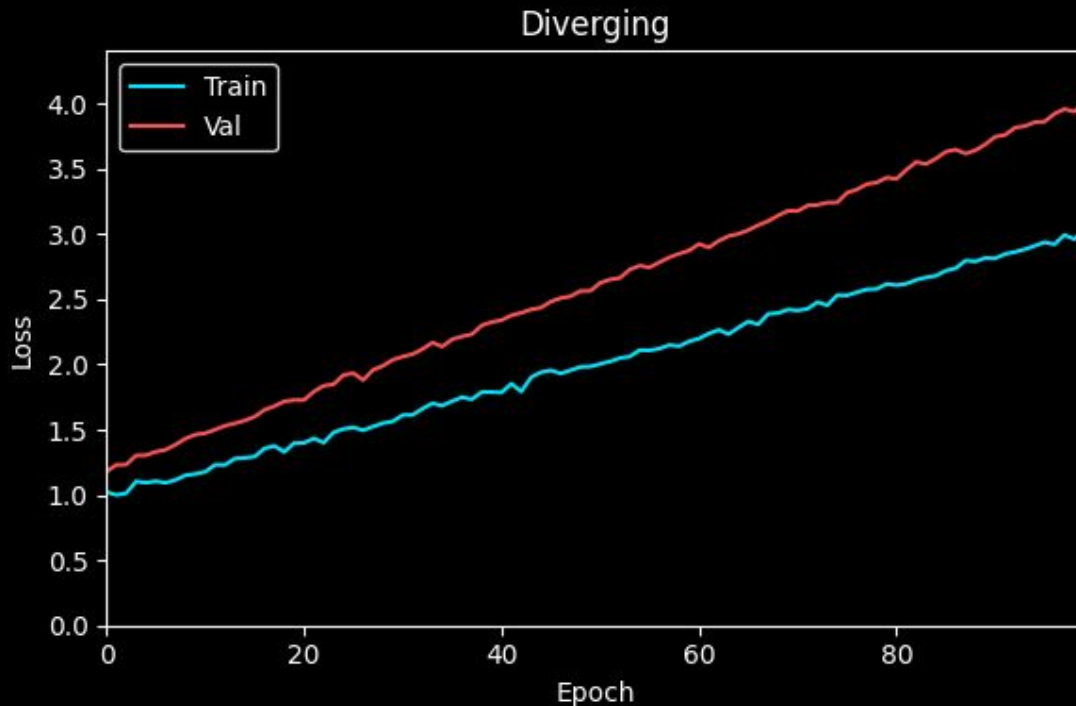
# Debugging Loss Curves

- Let's have a look at some plots...



# Case 1: Diverging

Why did this happen?

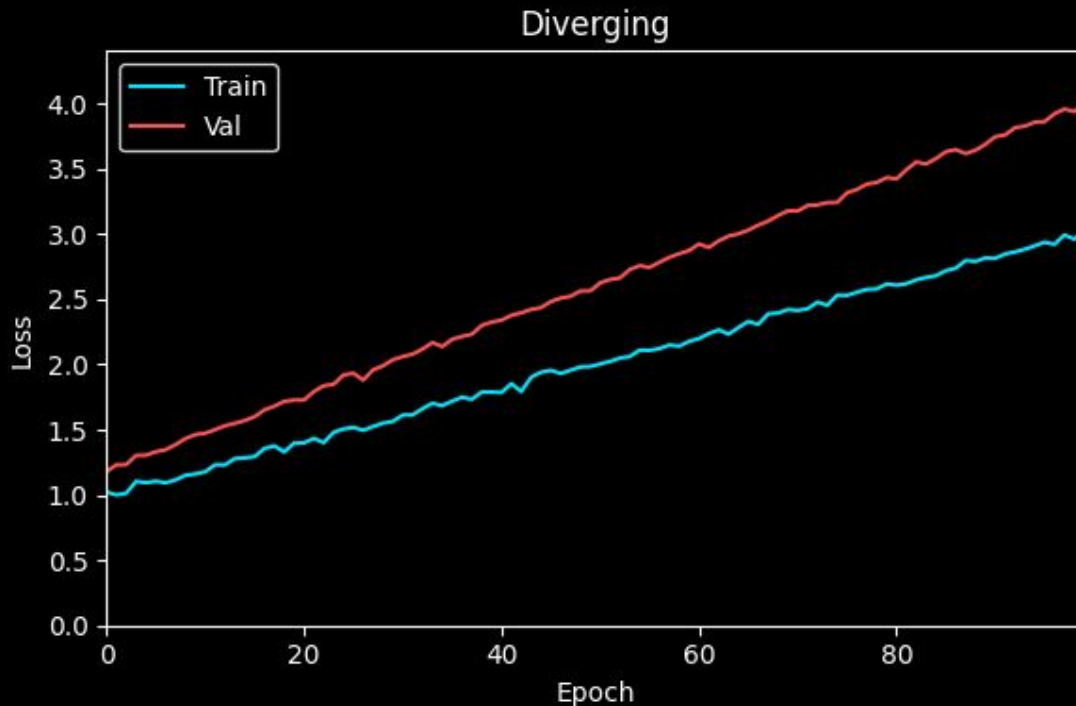




# Case 1: Diverging

Why did this happen?

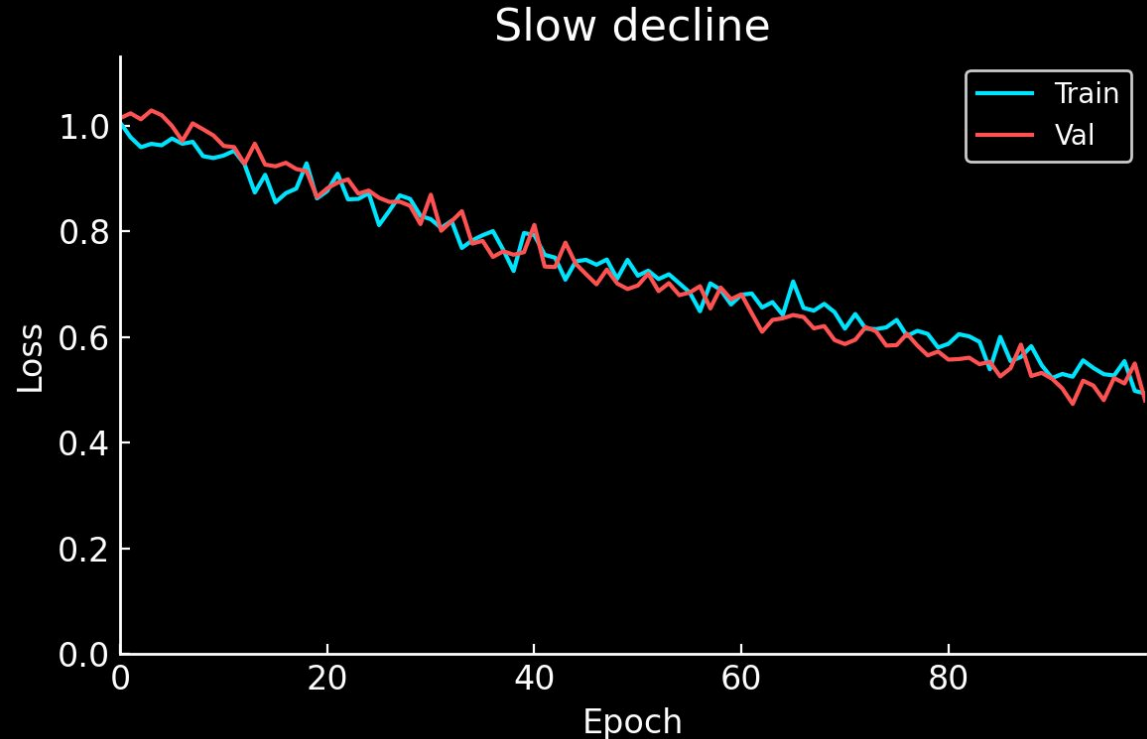
- High LR
- Exploding gradient
- Bad data
- No normalization





## Case 2: Slow Decline

Why did this happen?



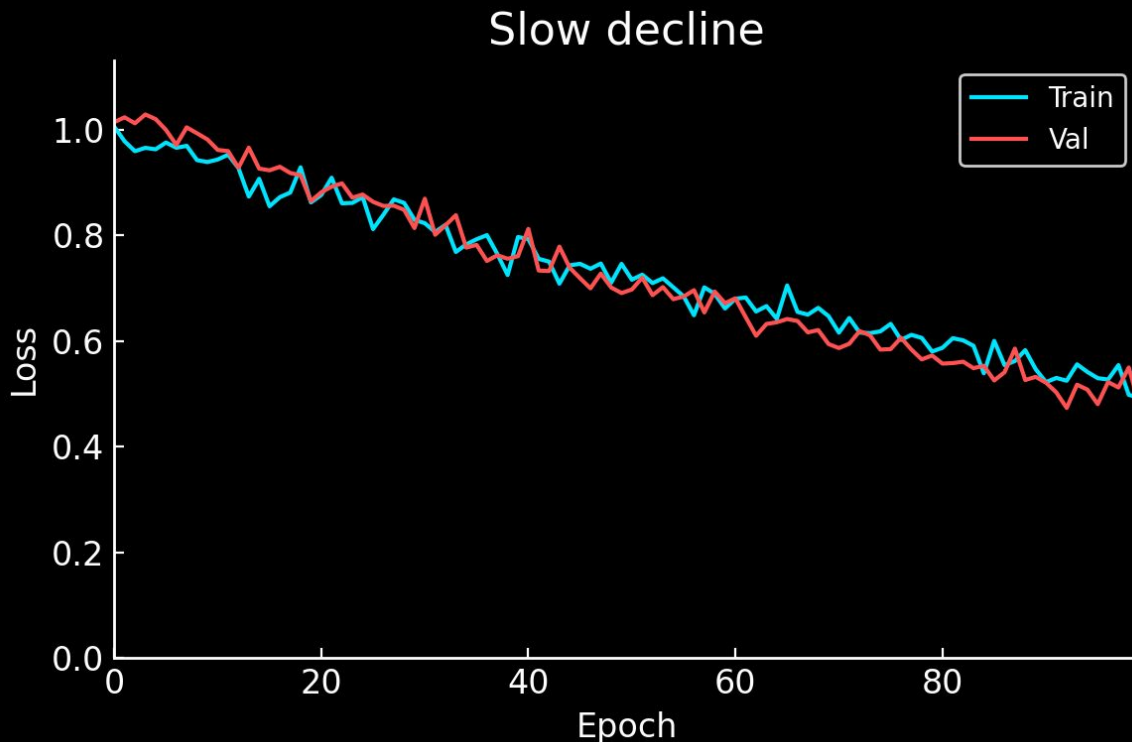




## Case 2: Slow Decline

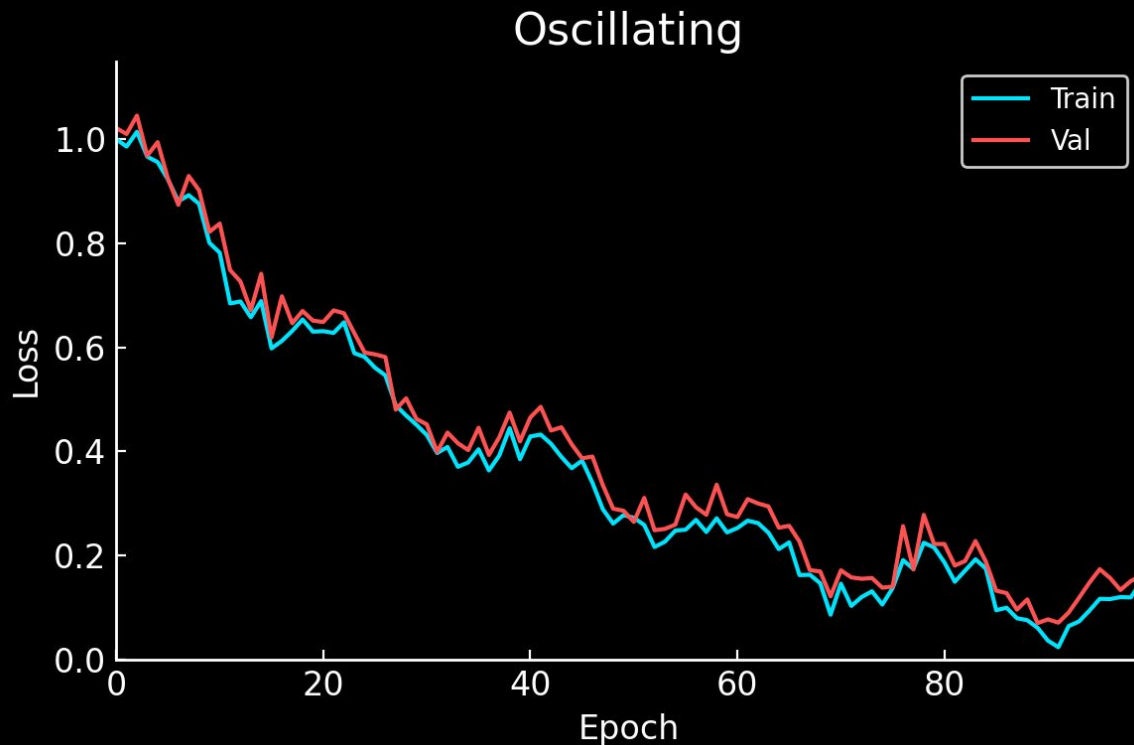
Why did this happen?

- Very small LR
- Vanishing gradients



# Case 3: Oscillating

Why did this happen?

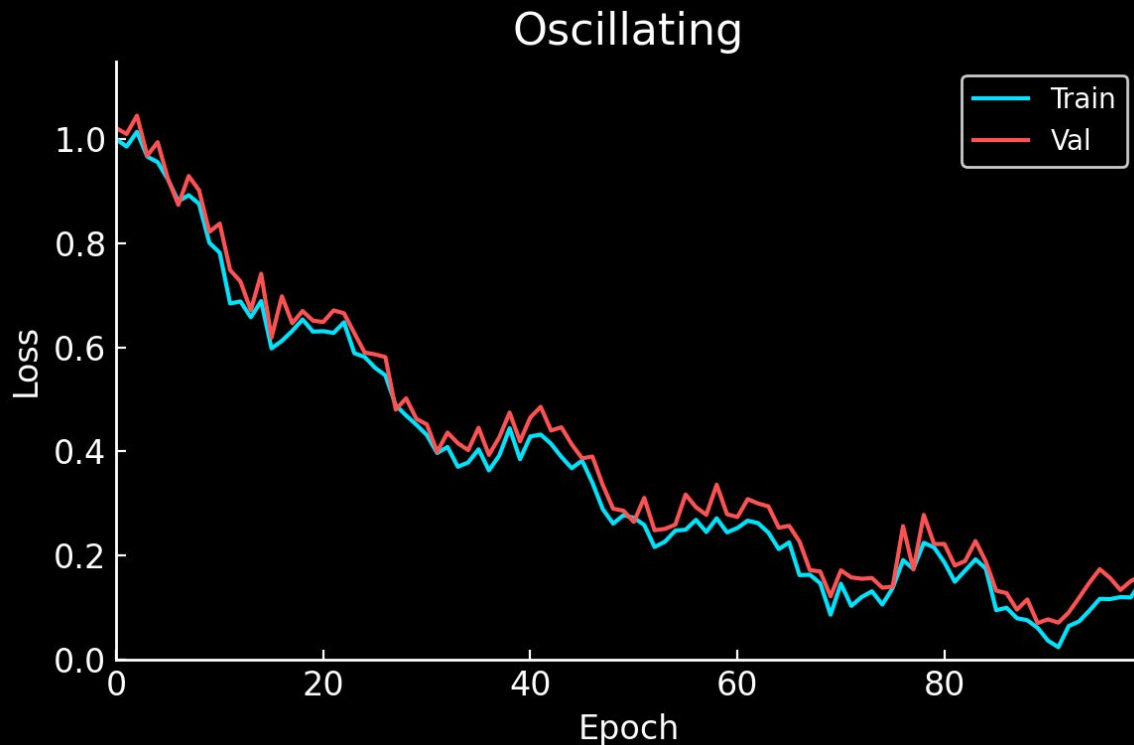




# Case 3: Oscillating

Why did this happen?

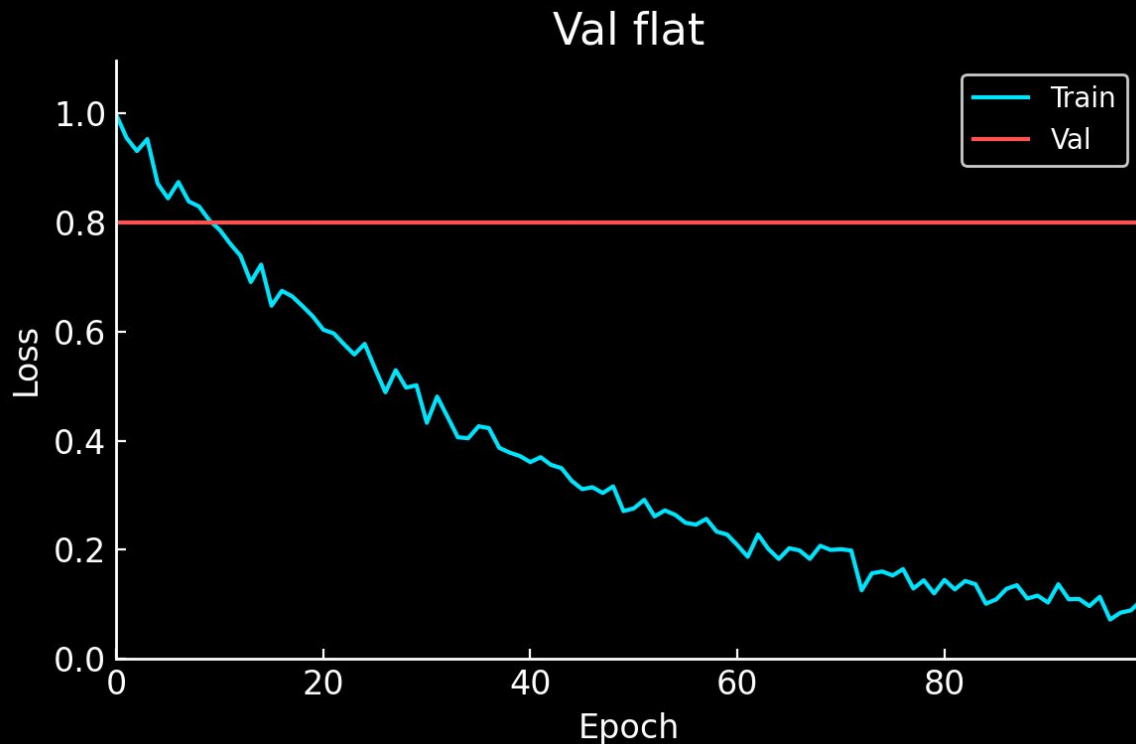
- High LR
- Small batch size
- Poor shuffle
- A lot of bad samples





## Case 4: Val flat

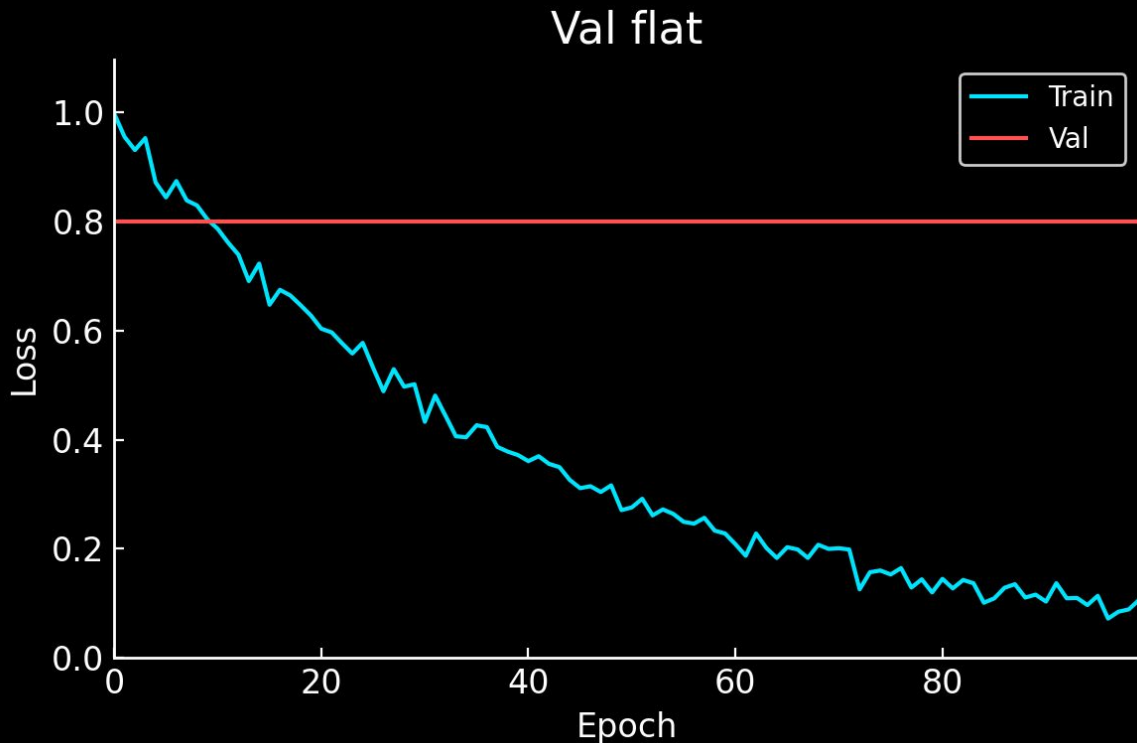
Why did this happen?



## Case 4: Val flat

Why did this happen?

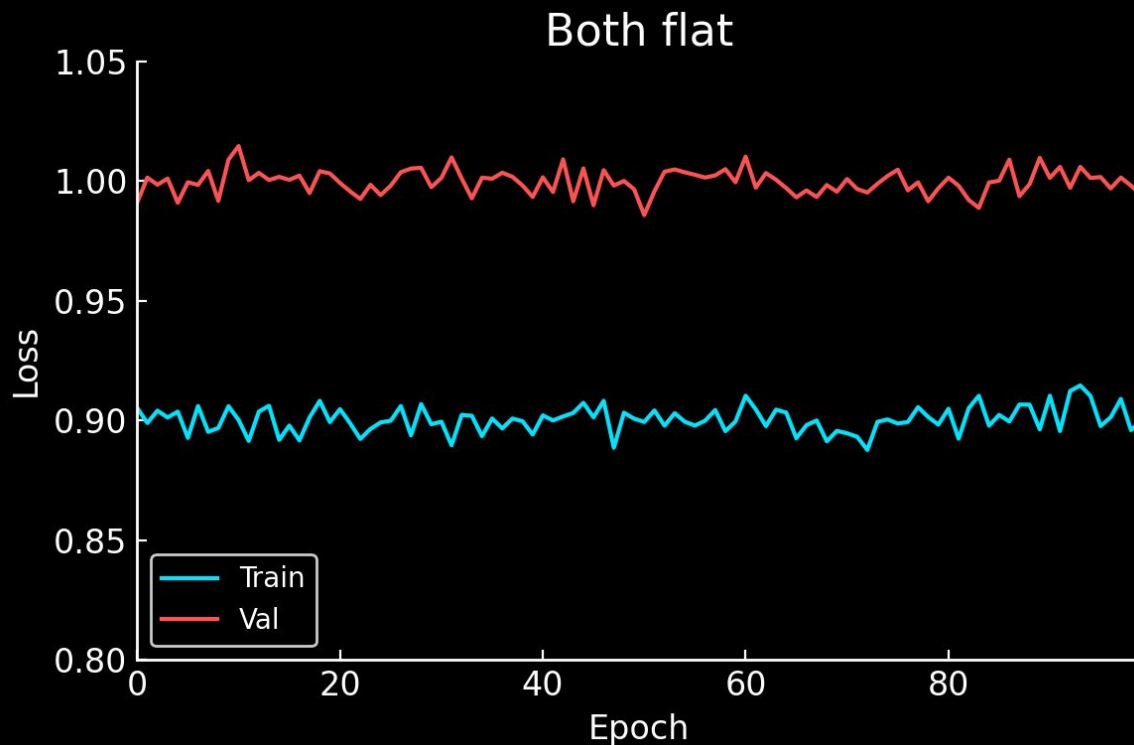
- Coding bug in data preparation (e.g. wrong labels)
- Very hard val (distribution shift)
- Significant imbalance





# Case 5: Both flat

Why did this happen?

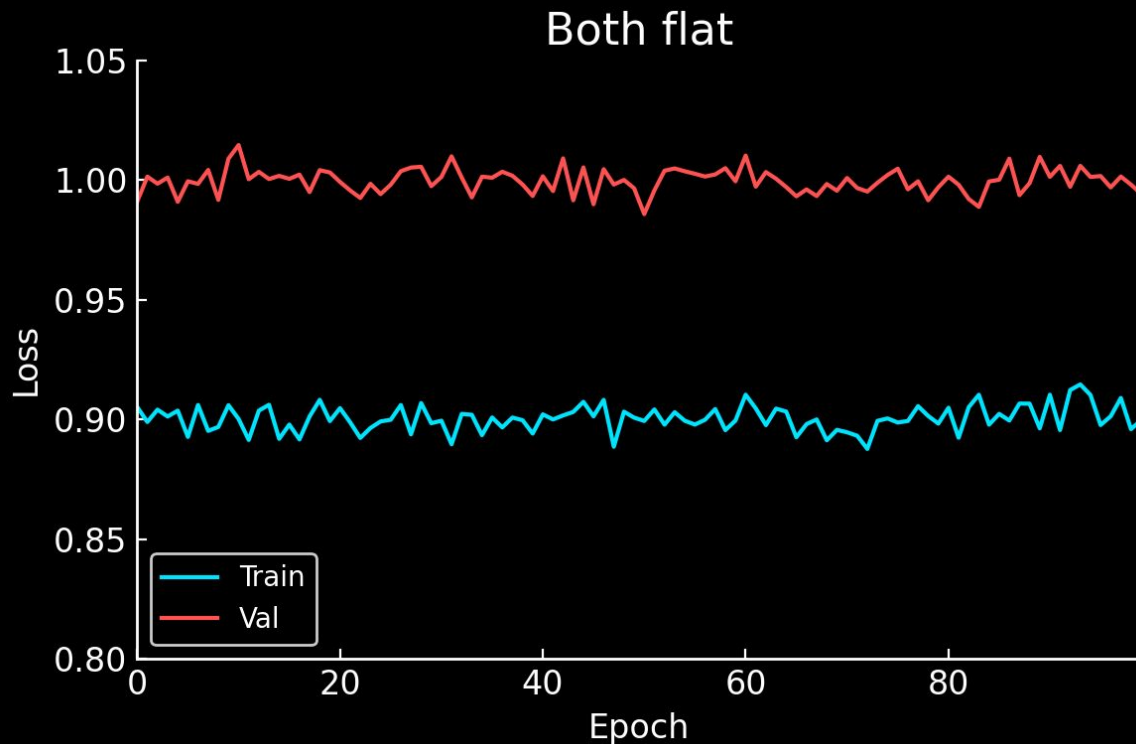




# Case 5: Both flat

Why did this happen?

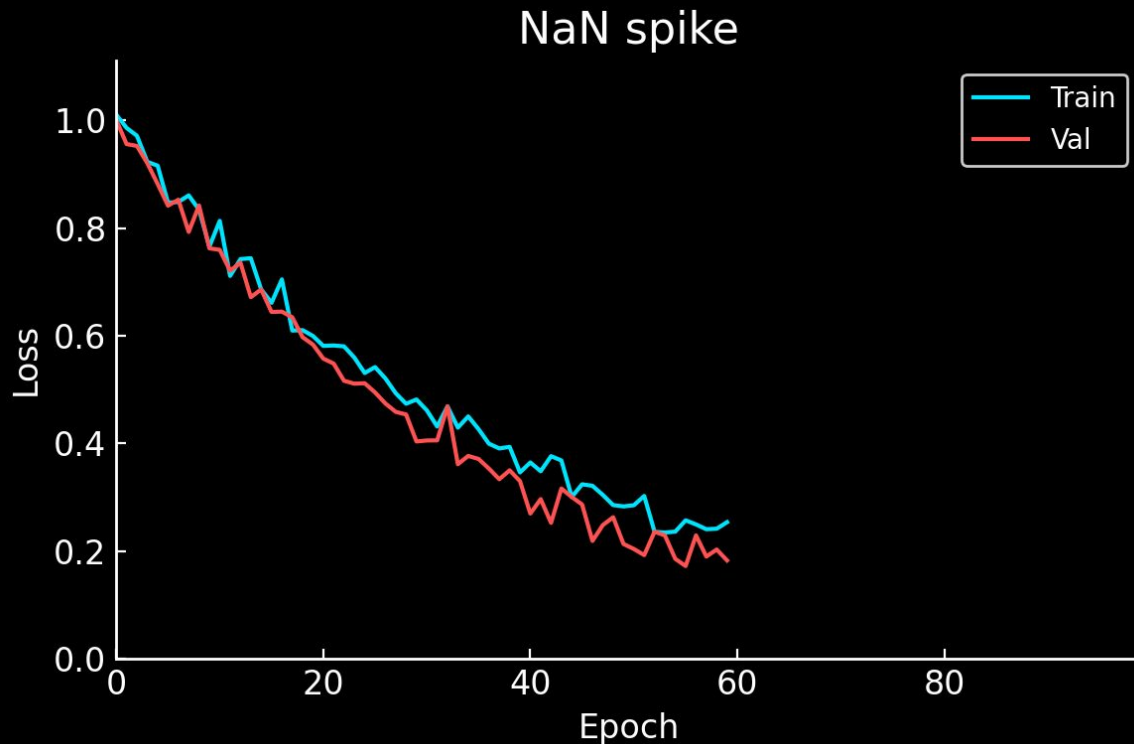
- Wrong loss
- Arch mismatch
- Bad labels
- Frozen grads
- Very small LR
- No normalization





# Case 6: NaN Loss

Why did this happen?

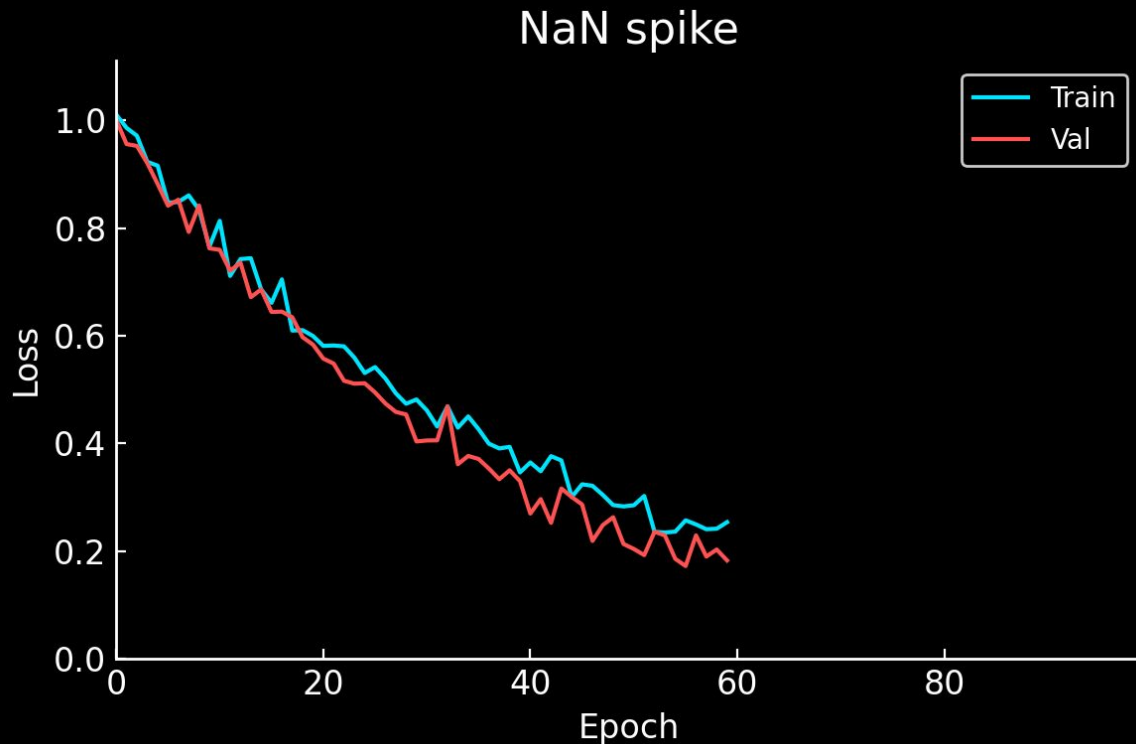




# Case 6: NaN Loss

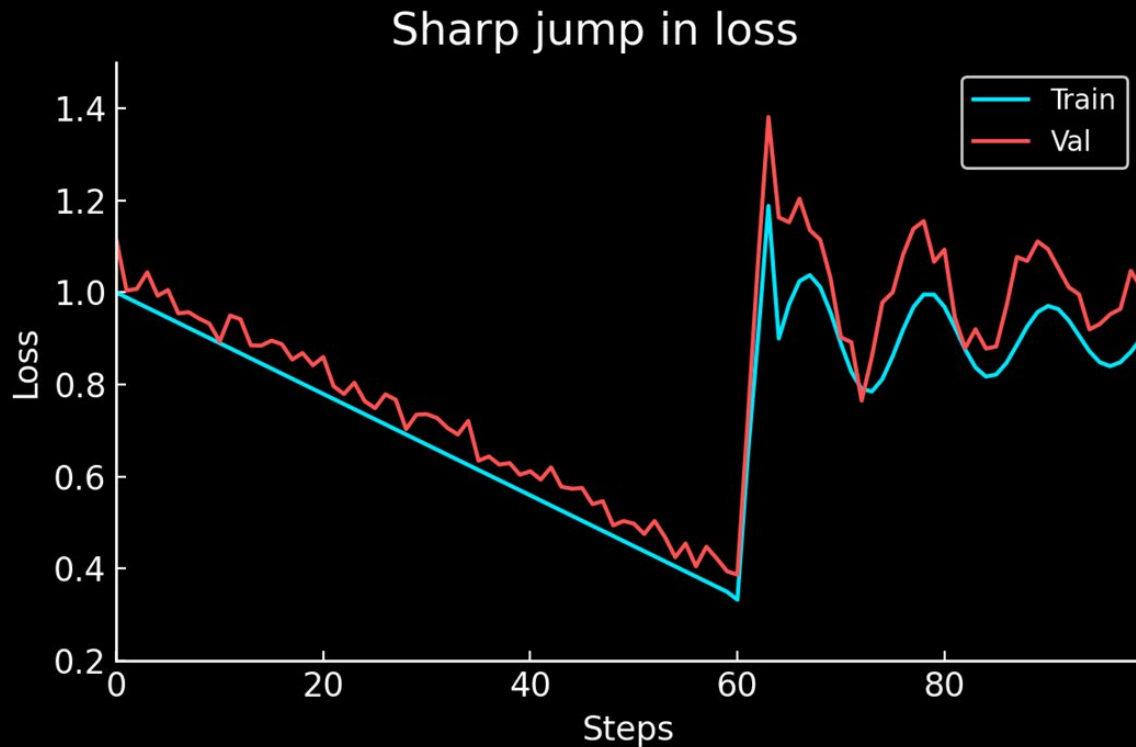
Why did this happen?

- Grad explode
- Log/Div 0



# Case 7: Sharp jump

Why did this happen?

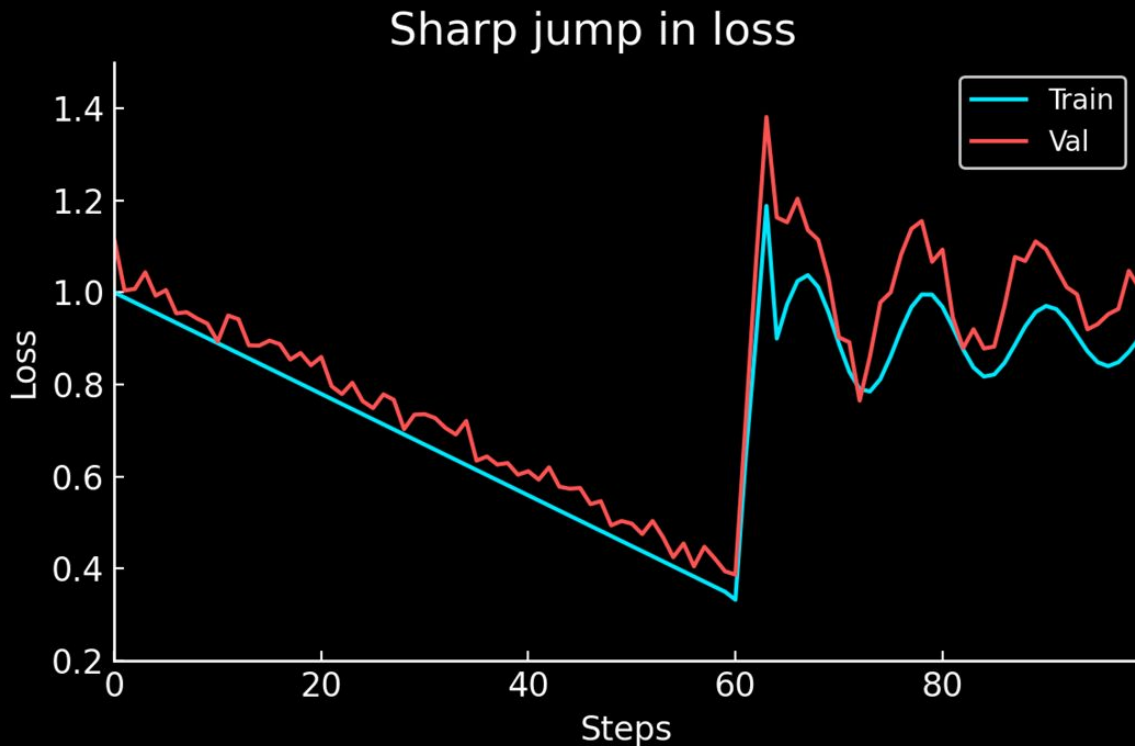




# Case 7: Sharp jump

Why did this happen?

- NaNs/Inf
- Outliers
- Poor shuffling



## Case 8: Val rises, train falls

Why did this happen?





## Case 8: Val rises, train falls

Why did this happen?

- Overfitting (Big Model, many epochs, few data, weak aug,...)



# Thanks for Attending!

Prepared By: Mohamed Eltayeb